LAB 7: Fuzzing for Input Validation Bugs

Objective:

To explore fuzz testing as a security and quality assurance technique by identifying and fixing bugs in the python script provided to you using the Hypothesis fuzzing library.

Duration:

60 minutes

Software Requirements:

- Python 3.8+

Learning Outcomes

By the end of this lab, students will:

- Understand the purpose and methodology of fuzz testing.
- Apply property-based fuzzing using Hypothesis.
- Detect and analyse bugs caused by edge-case inputs.
- Improve code robustness through defensive programming.

Introduction:

What is Fuzzing?

Fuzzing (or fuzz testing) is a **dynamic testing technique** where you feed **random, unexpected, or malformed data** to a program in order to:

- Discover bugs or crashes
- Uncover security vulnerabilities
- Ensure robustness against edge cases

Why Use Fuzzing?

- Manual tests often miss **rare edge cases**.
- Unit tests test what you think might go wrong.
- Fuzzers find what you **never imagined** could go wrong.

Example: What Can Go Wrong?

Consider this function:

```
def parse_int_list(s):
   return [int(x) for x in s.split(',')]
```

Works fine for:

```
parse_int_list("1,2,3")
```

But what about:

```
parse_int_list("1,,3")      # empty element
parse_int_list("one,two")   # ValueError
parse_int_list("")          # []
parse_int_list(None)        # TypeError
```

Fuzzers can generate such inputs automatically and expose hidden bugs like:

- Crashes (exceptions)
- Incorrect behavior
- Security issues (e.g., injection, DoS)

What is Hypothesis?

Hypothesis is a **property-based fuzz testing tool for Python**.

Instead of writing individual test cases, you describe **properties** of your function (e.g., it should not crash) and let Hypothesis generate dozens or hundreds of **random inputs** to verify those properties.

Example: Normal Unit Test vs Hypothesis

Traditional test:

```
def test_sanitize():
   assert sanitize_string("!hello!") == "hello"
```

Hypothesis Test:

```
from hypothesis import given, strategies as st
from utils import is_palindrome

@given(st.text() | st.none())
def test_is_palindrome_no_crash(s):
   try:
      is_palindrome(s)
   except Exception as e:
      assert False, f"is_palindrome crashed with: {e}"
```

You write the behaviour; Hypothesis tries to break it.

How Hypothesis Works

1. You write a test function using the @given decorator.
2. You specify **input types** (strategies) like st.text() or st.integers().
3. Hypothesis:
    - Automatically generates thousands of test cases.
    - Minimizes inputs to find the smallest failing case.
    - Logs failing inputs for debugging and replay.

Common Strategies in Hypothesis

Strategy	What it does
st.text()	Generates random Unicode strings
st.integers()	Random integers
st.lists(st.integers())	Lists of integers
st.booleans()	True/False values
st.dictionaries(keys, values)	Random dicts

You are now ready to execute this lab!

GOAL: Use fuzzing to test and fix real input validation bugs.

STEPS:

Step 1: Installing dependencies:

a. *pip install hypothesis* AND
b. *pip install pytest*

Step 2: Run the buggy application .py in the repo assigned to you (under Lab7):

Command: python processor.py

Provide the screenshot of the output

Step 3: Complete the code in test_processor.py using hypothesis:

Goal: Use Hypothesis to uncover more edge cases automatically for all three functions.

Step 4: Run the test_processor.py :

Command: pytest test_processor.py **Or**
python -m pytest test_processor.py (if pytest is installed but still showing not recognised)

Provide the screenshot of test cases failing.

Step 5: Fix the buggy application python script:

Goal: Harden the functions against all unexpected or invalid inputs, especially those discovered through fuzz testing.

Run the fixed processor.py – Command : python processor.py

Provide the screenshot of the output.

Step 6: Re-Run the test_processor.py wrt to fixed processor and take the ss of the output:

Command: pytest test_processor.py **Or**
python -m pytest test_processor.py (if pytest is installed but still showing not recognised)

Provide the screenshot of the output.

Deliverables:

1. **PDF Document with the screenshots**
2. **Fixed application python script**
3. **Updated test_processor.py code**